



ROB 2140

Siemens STRUCTURED TEXT LABS

John Sefcovic
Oakland Community College



For each lab, the named **Quick Watch** must be created and displayed during program execution for credit.

You may use **Copy** and **Paste** to make a copy of the text in this document to paste into the PLC editor.

DO NOT copy the instructions structures that must be dragged into the editor. These commands are noted in a **red box**.

Drag those instructions from the **Instruction Palette** into the PLC editor.

```
instruction text;  


instruction procedure;

  
instruction text;
```

TABLE OF CONTENTS

SIEMENS STRUCTURED TEXT LABS

LAB: SS1.....	1
LAB: WASHER STATION TIMER	6
LAB: TEMPERATURE CONVERSION with INDICATORS	10
EXAMPLE CODE	22

LAB: SS1

PROCESS MODEL: SS1 {last name} {first name}

START - STOP circuit for a single motor control with ON - OFF indicators.

PROGRAM LOCATION:

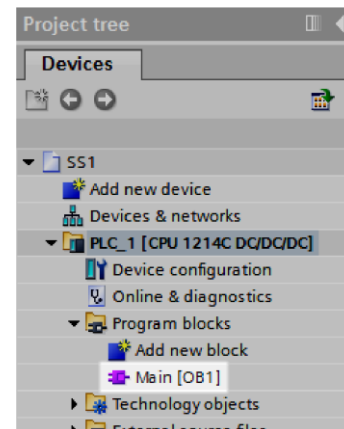
Siemens Folder
Languages Folder
Function Block Folder

FOLDER\FILE NAME: SS1

PROGRAM BLOCK:

Expand the Project Tree and select the **Main [OB1]**.

The blank editor will be displayed.



PLC TAGS

INPUT:

Address	PLC I/O Tag	Description
I0.0	STOP START	Pushbutton Momentary N.C.
I0.1	START STOP	Pushbutton Momentary N.O.

OUTPUT:

Address	PLC I/O Tag	Description
Q0.0	PL1_OFF	Pilot Light 1 OFF (Red)
Q0.1	PL2_ON	Pilot Light 1 ON (Green)
Q0.2	MOTOR	Motor Control

continue



TEXT ROUTINE

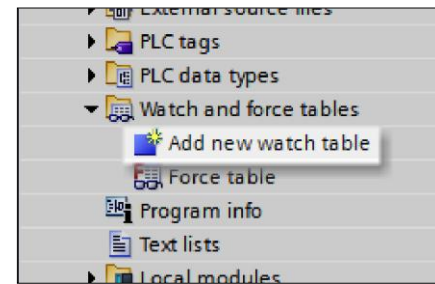
```
//START-STOP pushbuttons to turn On MOTOR
MOTOR := _____ AND ( _____ OR _____ );

//Indicators status based on MOTOR On or Off
PL1_OFF:= _____;
PL2_ON:= _____;
```

WATCH WINDOW

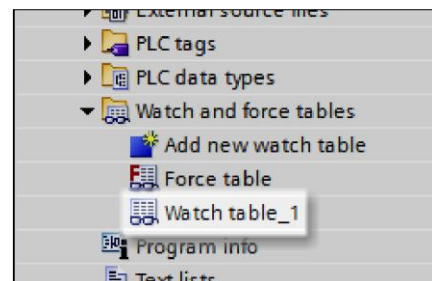
CREATE NEW WATCH WINDOW

From the **Project Tree**, select **Watch and force table** and then select **Add new watch table**.

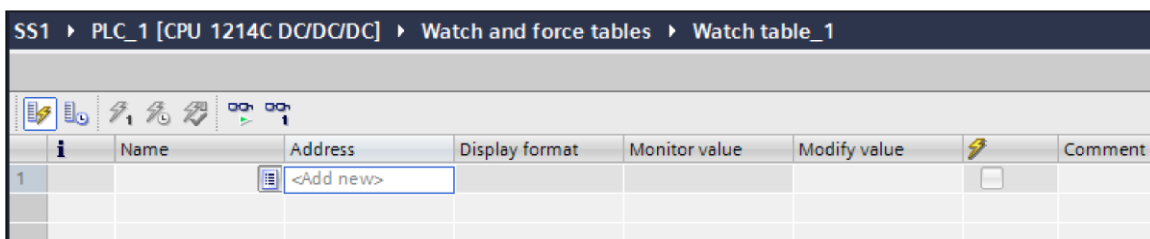


The default **Watch table_1** is created.

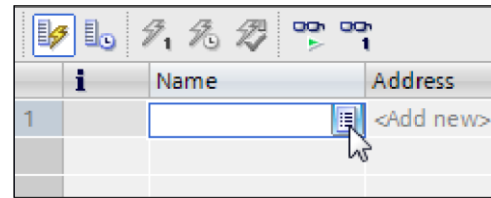
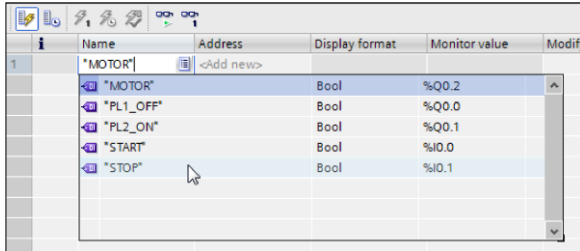
The name of the Watch Table may be changed by right clicking on the Project Tree and selecting Rename.



ADD TAGS TO WATCH WINDOW



To select tags, left click on **List** icon the **Name** field dropdown arrow and select the tag from the listing.



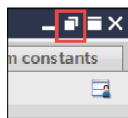
Completed Quick Watch.

	Name	Address	Display format
1	*STOP	%I0.1	Bool
2	*START	%I0.0	Bool
3	*MOTOR	%Q0.2	Bool
4	*PL1_OFF	%Q0.0	Bool
5	*PL2_ON	%Q0.1	Bool
6	<Add new>		

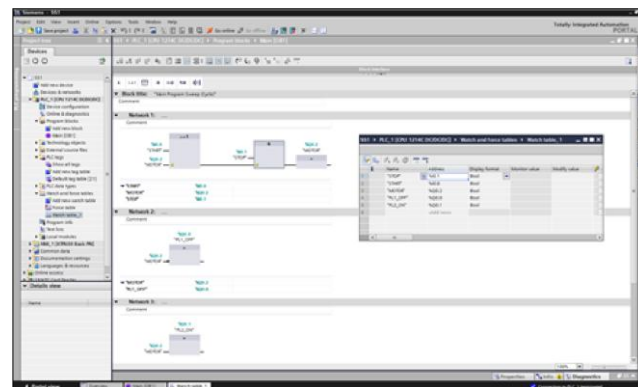
NOTE: If the Project is *not* saved, the Watch Window will be lost.

FLOAT WATCH WINDOW

Select the Float icon in the watch Window.



The Watch Window can then be resized and positioned for best viewing of multiple screens.



continue

MONITOR WATCH WINDOW

Do this step after the Project has been Downloaded to PLC.

In the Online Watch Window, select the **Monitor** icon.



	Name	Address	Display format	Monitor value	Modify value	
1	"STOP"	%I0.0	Bool	FALSE		☐
2	"START"	%I0.1	Bool	FALSE		☐
3	"PL1_OFF"	%Q0.0	Bool	TRUE		☐
4	"PL2_ON"	%Q0.1	Bool	FALSE		☐
5	"MOTOR"	%Q0.2	Bool	FALSE		☐
6	<Add new>					☐

The Monitored Watch Window shows the Boolean values as True or False with a small square as Green when True.

	Name	Address	Display format	Monitor value	Modify value	
1	"STOP"	%I0.0	Bool	FALSE		☐
2	"START"	%I0.1	Bool	FALSE		☐
3	"PL1_OFF"	%Q0.0	Bool	TRUE		☐
4	"PL2_ON"	%Q0.1	Bool	FALSE		☐
5	"MOTOR"	%Q0.2	Bool	FALSE		☐
6	<Add new>					☐

DOWNLOAD

Download the program to the PLC.

There is no HMI.

Select the **Monitor** icon for the PLC program.

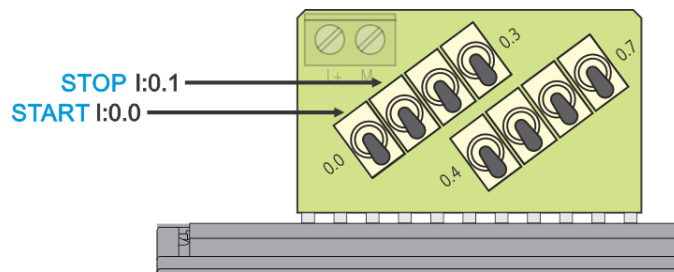


In the Online Watch Window, select the **Monitor** icon.



continue

TESTING



Initial Conditions.

- Turn the switch **I0.1** STOP to the **up** position.
- The STOP Monitor value is set to **TRUE**.
- Turn the switch **I0.0** START to the **down** position.
- The START Monitor value is set to **FALSE**.
- The PL1_OFF indicator is On; the Monitor value is set to **TRUE**.
- The PL2_ON indicator is Off, the Monitor value is reset to **FALSE**.

Pressing and Releasing the START pushbutton.

- Turn the switch **I0.0** START to the **up** position.
- The START Monitor value is set to **TRUE**.
- Turn the switch **I0.0** START to the **down** position.
- The START Monitor value is set to **FALSE**.
- MOTOR will turn On, the Monitor value is set to **TRUE**.
- The PL1_OFF indicator is Off, the Monitor value is reset to **FALSE**.
- The PL2_ON indicator is On, the Monitor value is set to **TRUE**.

Pressing the STOP pushbutton.

- Turn the **switch I0.1** STOP to the **down** position.
- The STOP Monitor value is set to **FALSE**.
- MOTOR will turn Off, the Monitor value is reset to **FALSE**.
- The PL1_OFF indicator is On; the Monitor value is set to **TRUE**.
- The PL2_ON indicator is Off, the Monitor value is reset to **FALSE**.



LAB: WASHER STATION TIMER

PROCESS MODEL: WASH_TON {last name} {first name}

OBJECTIVE: Create structured text program using the timer on instruction.

SOFTWARE: Siemens TIA

Create a program that when a momentary normally open pushbutton is pressed and held, two clamp solenoids will be energized. A normally open limit switch for each clamp will be set when that clamp is in position. When the two clamps are in position, a spray wash will clean the part for ten seconds. Releasing the pushbutton will turn off the spray wash and clamps.

PROGRAM LOCATION:

Siemens Folder

Languages Folder

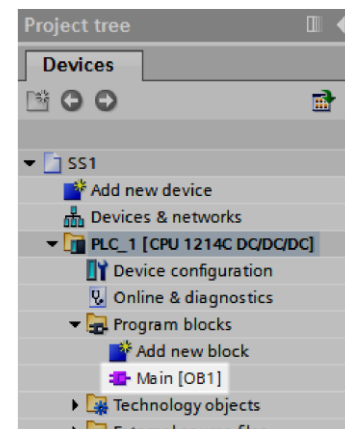
Function Block Folder

FOLDER\FILE NAME: WASH_TON

PROGRAM BLOCK:

Expand the Project Tree and select the **Main [OB1]**.

The blank editor will be displayed.



PLC TAGS

INPUT:

Address	PLC I/O Tag	Description
I0.0	PUSHBUTTON	Pushbutton Wash Cycle
I0.1	SWITCH_CLAMP1	Limit Switch Clamp 1
I0.2	SWITCH_CLAMP2	Limit Switch Clamp 2

OUTPUT:

Address	PLC I/O Tag	Description
Q0.0	SOLENOID_CLAMP1	Clamp 1 Solenoid
Q0.1	SOLENOID_CLAMP2	Clamp 2 Solenoid
Q0.2	WASH_VALVE	Spray Wash Valve



Notes on Program:

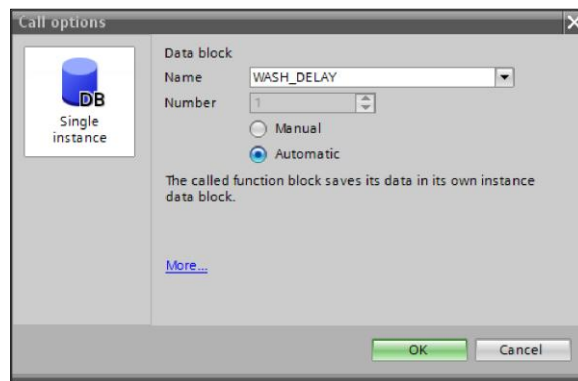
Drag from the **Instruction Folder** the **TON**.

After added Timer ON instruction the timer's Data block dialog appears.

Change the **Name** to WASH_DELAY.

Delete unused parameters.

TON instruction, **Timer operations** folder.



TEXT ROUTINE

Using the pushbutton, clamp switches, and timer done to control wash valve.

When entering the Timer Tag, type the " (quote) to open the Tag Listing. Then type the . (period) to open the timer Tag Listing.

```
//START pushbuttons to turn On CLAMP solenoids
```

```
IF _____ THEN
```

```
"SOLENOID_CLAMP1" := _____;
```

```
"SOLENOID_CLAMP2" := _____;
```

```
ELSE;
```

```
"SOLENOID_CLAMP1" := _____;
```

```
"SOLENOID_CLAMP2" := _____;
```

```
END_IF;
```

```
//Timer structure for time to wash when PUSHBUTTON, CLAMP 1 and CLAMP
```

```
//2 switches are closed
```

```
"WASH_DELAY".TON(IN:= _____ & _____ & _____,  
PT:=T# _____ ms);
```

```
//Turn On WASH valve when PUSHBUTTON, CLAMP 1 and CLAMP 2 switches are
```

```
//closed and timer in not complete
```

```
IF _____ & _____ & _____ & _____ "_____"._____ THEN  
_____ := _____;
```

```
ELSE;
```

```
"WASH_VALVE" := _____;
```

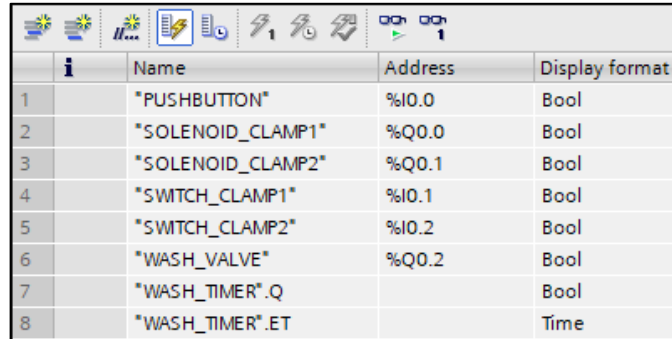
```
END_IF;
```



WATCH WINDOW

See SS1 for similar details on Watch Window creation and usage, pages 2 to 4.

Completed Quick Watch.



The screenshot shows the Watch Window toolbar with icons for Watch, Quick Watch, and other functions. Below the toolbar is a table with the following data:

	Name	Address	Display format
1	"PUSHBUTTON"	%I0.0	Bool
2	"SOLENOID_CLAMP1"	%Q0.0	Bool
3	"SOLENOID_CLAMP2"	%Q0.1	Bool
4	"SWITCH_CLAMP1"	%I0.1	Bool
5	"SWITCH_CLAMP2"	%I0.2	Bool
6	"WASH_VALVE"	%Q0.2	Bool
7	"WASH_TIMER".Q		Bool
8	"WASH_TIMER".ET		Time

To enter the Timer, select from the Listing Icon the **"WASH_TIMER"** tag. Then type a Period . for the timer's element Tags.

NOTE: If the Project is *not saved*, the Watch Window will be lost.

DOWNLOAD

Download the program to the PLC.

There is no HMI.

Select the **Monitor** icon for the PLC program.

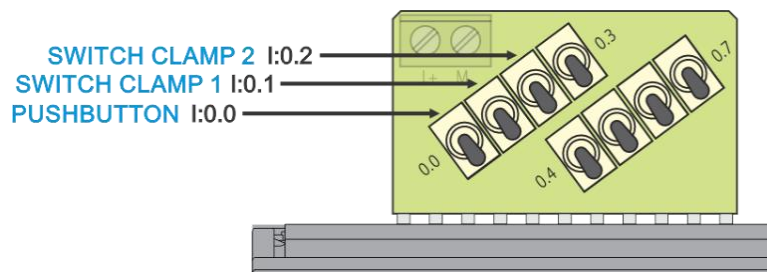


In the Online Watch Window, select the **Monitor** icon.



continue

TESTING



Pressing the PUSHBUTTON.

- Turn the switch **I0.0** PUSHBUTTON to the **up** position.
- The PUSHBUTTON Monitor value is set to **TRUE**.
- The SOLENOID_CLAMP1 will turn On, the Monitor values are set to **TRUE**.
- The SOLENOID_CLAMP2 will turn On, the Monitor values are set to **TRUE**.
- Turn the switch **I0.1** SWITCH_CLAMP1 to the **up** position.
- The SWITCH_CLAMP1 turns On, the Monitor value is set to **TRUE**.
- Turn the switch **I0.2** SWITCH_CLAMP2 to the **up** position.
- The SWITCH_CLAMP2 turns On, the Monitor value is set to **TRUE**.
- The WASH_VALVE turns On, the Monitor value is set to **TRUE**.
- WASH_DELAY elapsing increments.
- After 10 Second, the WASH_VALVE turns Off, the Monitor value is **FALSE**.

Releasing the PUSHBUTTON.

- Turn the switch **I0.0** PUSHBUTTON to the **down** position.
- Changes PUSHBUTTON Monitor value, resets to **FALSE**.
- Turns Off the SOLENOIDS_CLAMP1, the Monitor values are reset to **FALSE**.
- Turns Off the SOLENOIDS_CLAMP2, the Monitor values are reset to **FALSE**.
- Turn the switch **I0.1** SWITCH_CLAMP1 to the **down** position.
- The SWITCH_CLAMP1 turns Off, the Monitor value is reset to **FALSE**.
- Turn the switch **I0.2** SWITCH_CLAMP2 to the **down** position
- The SWITCH_CLAMP2 turns Off, the Monitor value is set to **FALSE**.



LAB: TEMPERATURE CONVERSION with INDICATORS**PROCESS MODEL:** Text_CONVERT {last name} {first name}**OBJECTIVE:** Create ladder structure and assign data tags.**SOFTWARE:** Siemens TIA

Create a program that will convert a Celsius temperature to a Fahrenheit temperature value. The maximum Celsius value will be 100 degrees.

$$\text{Equation: } F = (C \times 9) / 5 + 32$$

The project will use either hardwired or HMI input pushbuttons. A maintenance screen will set a signal to disable the use of the hardwired pushbuttons.

PLC Tags**INPUT:**

Address	PLC I/O Tag	Comments
I0.0	CONVERT_PB	Convert Temperature N. O. pushbutton
I0.3	RESET_PB	Data Error Reset, N. O. pushbutton

OUTPUT:

Address	PLC I/O Tag	Comments
Q0.0	ERROR_DUP_VAL	Error Indicator
Q0.1	ERROR_OVER_VAL	Data Error, Over Value
Q0.2	LL_INDC	Low Low Indicator
Q0.3	L_INDC	Low Indicator
Q0.4	M_INDC	Mid Indicator
Q0.5	H_INDC	High Indicator
Q0.6	HH_INDC	High High Indicator

continue

Program Structure

Program Function routine:

- * Block_Indicators

Program Function Block routines

- @ Block_Input_Control
- @ Block_Convert

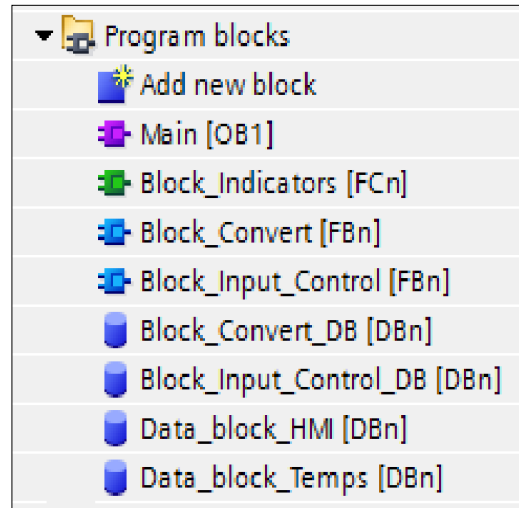
Function Blocks Data Blocks will be created when Function Block call added to OB1.

Function Block Data Blocks

Block_Convert_DB
Block_Input_Control_DB

Global Data Blocks:

Data_block_HMI
Data_block_Temps



n = automatically assigned block number

Create the Global Data Blocks and rename. Add the following Tags.

Name of Data Block: **Data_block_HMI**

DATA BLOCK:

Data Block Tag	Data Type	Comments
HMI_CONVERT_PB	Boolean	HMI Convert Button
HMI_RESET_PB	Boolean	HMI Reset Button
HMI_ENABLE_DISABLE	Boolean	HMI Enable/Disable Button

continue



Name of Data Block: **Data_block_Temps**

DATA BLOCK:

Data Block Tag	Data Type	Comments
C_TEMP	DINT	Input of Celsius Temperature
C_TEMP_STORAGE	DINT	Storage of Celsius Temperature
F_TEMP	DINT	Converted Fahrenheit Result

TEXT ROUTINES

LOGIC FOR FUNCTION BLOCK CONVERT

Create the **Function Block**.

Name of Function Block: **Block_Convert**

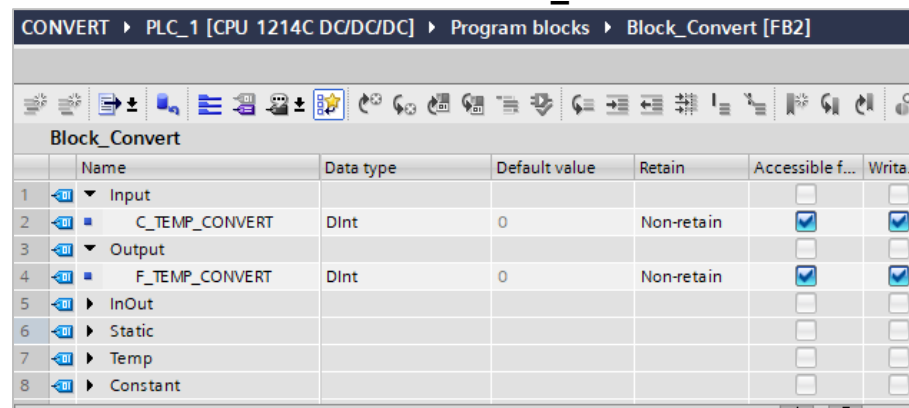
Use the Math function instructions to convert the Celsius value to a Fahrenheit value.

INTERFACE for Function Block - **Block_Convert**:

Input Tag	Data Type	Comment
C_TEMP_CONVERT	DINT	Celsius Temperature to Function Block

Output Tag	Data Type	Comment
F_TEMP_CONVERT	DINT	Fahrenheit Temperature from Function Block

Interface for the Function Block **Block_Convert**.



Block_Convert						
	Name	Data type	Default value	Retain	Accessible f...	Writa...
1	Input					
2	C_TEMP_CONVERT	Dint	0	Non-retain	☒	☒
3	Output					
4	F_TEMP_CONVERT	Dint	0	Non-retain	☒	☒
5	InOut					
6	Static					
7	Temp					
8	Constant					



Equation: $F = (C(9)/5) + 32$

To open the Tag Listing for type the # (hash).

```
//Conversion of Celsius value to Fahrenheit temperature.  
#F_TEMP_CONVERT := ( #_____ * 9) / 5 + 32;
```

LOGIC FOR FUNCTION BLOCK INPUT_CONTROL

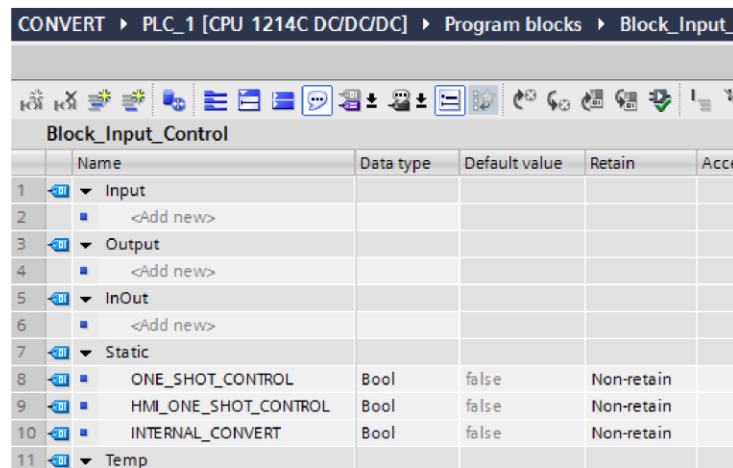
Create the **Function Block**.

Name of Function Block: **Block_Input_Control**

INTERFACE for Block_Input_Control:

Static	Data Type	Comments
ONE_SHOT_CONTROL	Boolean	One Shot, Hardwired Panel
HMI_ONE_SHOT_CONTROL	Boolean	One Shot, HMI
INTERNAL_CONVERT	Boolean	Internal Control, Enable Convert

*Interface for the Function Block **Block_Input_Control**.*



The screenshot shows the 'Block_Input_Control' interface in the HW Config software. The interface is organized into a table with columns: Name, Data type, Default value, Retain, and Access. The table lists the following static variables:

	Name	Data type	Default value	Retain	Access
1	Input				
2	<Add new>				
3	Output				
4	<Add new>				
5	InOut				
6	<Add new>				
7	Static				
8	ONE_SHOT_CONTROL	Bool	false	Non-retain	
9	HMI_ONE_SHOT_CONTROL	Bool	false	Non-retain	
10	INTERNAL_CONVERT	Bool	false	Non-retain	
11	Temp				

INFORMATION - R_TRIG INSTRUCTION

NOTE: For the one-shot instruction, R_TRIG, Set Tag on Positive Signal Edge.

Bit logic operations

R_TRIG

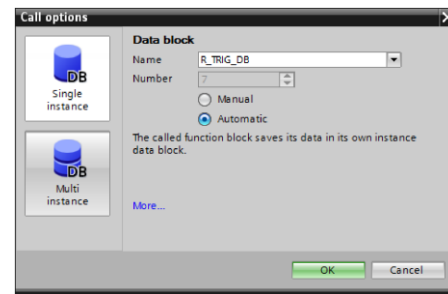
When the One Shot instruction is dragged onto the editor a call dialog opens.



In System blocks and Program resources a protected Function Block and a Data Block is created for each instance of the instruction.

Name of Data Blocks:

R_TRIG_SWITCH
R_TRIG_HMI



INFORMATION – BLOCK CALL

Example of dragging a Call.

This example uses Block_Indicators, use the Call required.



INFORMATION - LOCAL TAGS

To open the **Tag Listing** for type the # (hash).

When entering the **Tag** from a **Data Block**, type the " (quote) to open the **Tag Listing**. Then type the . (period) to open the **Tag Listing** for the Data Block.

```
//One shot for CONVERT PUSHBUTTON for ONE SHOT CONTROL.
```

```
"R_TRIG_SWITCH" (CLK:= _____,  
Q=># _____) ;
```

```
//One shot for HMI CONVERT PUSHBUTTON for HMI ONE SHOT CONTROL.
```

```
"R_TRIG_HMI" (CLK:= " _____ ". _____,  
Q=># _____) ;
```

```
//One shot for conversion.
```

```
IF # _____ OR # _____ THEN  
    #INTERNAL_CONVERT := _____;  
ELSE  
    #INTERNAL_CONVERT := _____;  
END_IF;
```



```

//If no previous Errors, check for Duplicate or Over Values
//and set alarms.
IF _____ AND NOT ( _____ ) AND NOT ( _____ )
THEN
    "ERROR_DUP_VAL" := " _____ " . _____ =
    " _____ " . _____ ;
    "ERROR_OVER_VAL" := " _____ " . _____ > _____ ;
END_IF;

//Error RESET PUSHBUTTON and clearing alarms.
IF _____ OR " _____ " . _____ THEN
    "ERROR_DUP_VAL" := _____ ;
    "ERROR_OVER_VAL" := _____ ;
END_IF;

/*If no errors, store the Celsius value and call the conversion
routine with the Celsius temperature to return the Fahrenheit
temperature.*/
IF # _____ AND NOT ( _____ ) AND NOT ( _____ ) THEN
    " _____ " . _____ := " _____ " . _____ ;
    "Block_Convert_DB" (C_TEMP_CONVERT := " _____ " . _____ ,
                        F_TEMP_CONVERT => " _____ " . _____ ) ;
END_IF;

```

LOGIC FOR FUNCTION INDICATORS

Create the **Function**.

Name of Function: **Block_Indicators**

The In Range instruction for the range indicators **must** be used.

If a conversion is valid, indicators will show the converted temperature is within a LOW LOW, LOW, MID, HIGH, or HIGH HIGH range. Use the IN_RANGE instructions.

Ranges:	Low Low	32 to 49
	Low	50 to 99
	Mid	100 to 149
	High	150 to 199
	High High	200 to 212



```

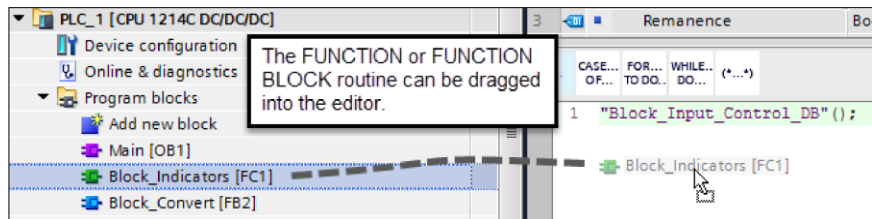
//Select Case to set range indicator.
CASE "_____"._____ OF
  32..49:
    "LL_INDC" := ____;
    "L_INDC" := ____;
    "M_INDC" := ____;
    "H_INDC" := ____;
    "HH_INDC" := ____;
  50..99:
    "LL_INDC" := ____;
    "L_INDC" := ____;
    "M_INDC" := ____;
    "H_INDC" := ____;
    "HH_INDC" := ____;
  ____..____:
    "LL_INDC" := ____;
    "L_INDC" := ____;
    "M_INDC" := ____;
    "H_INDC" := ____;
    "HH_INDC" := ____;
  ____..____:
    "LL_INDC" := ____;
    "L_INDC" := ____;
    "M_INDC" := ____;
    "H_INDC" := ____;
    "HH_INDC" := ____;
  ____..____:
    "LL_INDC" := ____;
    "L_INDC" := ____;
    "M_INDC" := ____;
    "H_INDC" := ____;
    "HH_INDC" := ____;
END_CASE;

```

LOGIC FOR ORGANIZATION BLOCK Main [OB1]

Example of dragging a Call.

This example uses Block_Indicators, use the Call required.



The Function Block **Block_Input_Control** for the conversion is called unconditionally.

The Function **Block_Indicators** for the indicators is called unconditionally.

WATCH WINDOW

See SS1 for similar details on Watch Window creation and usage, pages 2 and 3.

Completed Quick Watch.

	Name	Address	Display format
1	"Data_block_Temps".C_TEMP		DEC_signed
2	"Data_block_Temps".F_TEMP		DEC_signed
3	"HH_INDC"	%Q0.6	Bool
4	"H_INDC"	%Q0.5	Bool
5	"M_INDC"	%Q0.4	Bool
6	"L_INDC"	%Q0.3	Bool
7	"LL_INDC"	%Q0.2	Bool
8	"CONVERT_FB"	%I0.0	Bool
9	"ERROR_DUP_VAL"	%Q0.0	Bool
10	"ERROR_OVER_VAL"	%Q0.1	Bool

NOTE: If Project is *not saved*, the Watch Window will be lost.

DOWNLOAD

Download the program to the PLC.

Select the **Monitor** icon for the PLC program.



In the Online Watch Window, select the **Monitor** icon.



Download the **HMI**.



TESTING

DISABLE HARDWARE PANEL

1. From the MAIN HMI screen select the **MAINTENANCE** panel.

A rectangular button with the word "MAINTENANCE" in black capital letters.

2. Select **HARDWIRED PANEL DISABLED**.

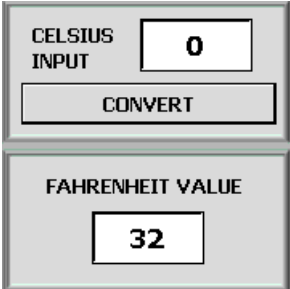
A rectangular button with the text "HARDWIRED PANEL DISABLED" in black capital letters.

TESTING TEMPERATURE CONVERSION

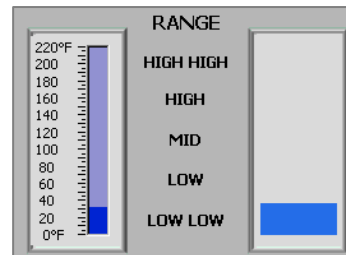
1. From the **MAINTENANCE** screen go to the **INPUT** screen.

A rectangular button with the word "INPUT" in black capital letters.

2. Enter a Celsius value of 0°.
3. Toggle (ON-OFF) the **CONVERT** pushbutton
4. Check for the correct converted temperature, *should be 32° F*.

The HMI screen for temperature conversion. It has two main sections. The top section is labeled "CELSIUS INPUT" and contains a text box with the number "0" and a "CONVERT" pushbutton below it. The bottom section is labeled "FAHRENHEIT VALUE" and contains a text box with the number "32".

5. Go to the **RANGE** screen and check for the correct **Range Indicator**.

A rectangular button with the word "RANGE" in black capital letters.

6. Go to the **INPUT** screen.

A rectangular button with the word "INPUT" in black capital letters.

Repeat the steps 4, 5, and 6 with the other Celsius values into C_TEMP to test the Range Indicators.

Celsius	Fahrenheit	Range
0°	32°	Low Low
30°	86°	Low
60°	140°	Mid
80°	176°	High
100°	212°	High High

continue

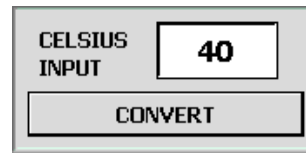


TEST DUPLICATE ERROR

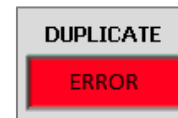
1. Go to the **INPUT** screen.



2. Enter a Celsius value of 40.
3. Press the **CONVERT** pushbutton.
4. Again, Press the **CONVERT** pushbutton.



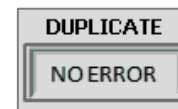
ERROR DUPLICATE VALUE indicator should be ON.



Toggle (ON-OFF) the **RESET** pushbutton.

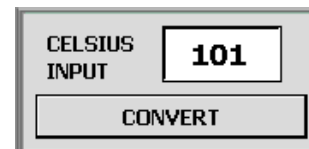


ERROR DUPLICATE VALUE indicator should be OFF.

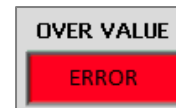


TEST OVER TEMPRATURE VALUE ERROR

1. Enter a Celsius value of 101°.
2. Toggle (ON-OFF) the **CONVERT** pushbutton.



ERROR OVER VALUE indicator should be ON.



3. Toggle (ON-OFF) the **RESET** pushbutton.



ERROR OVER VALUE indicator should be OFF.

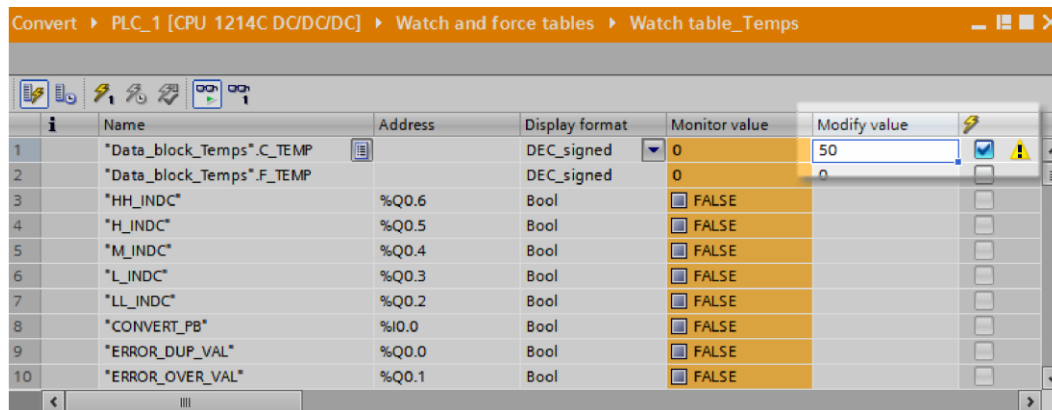


continue



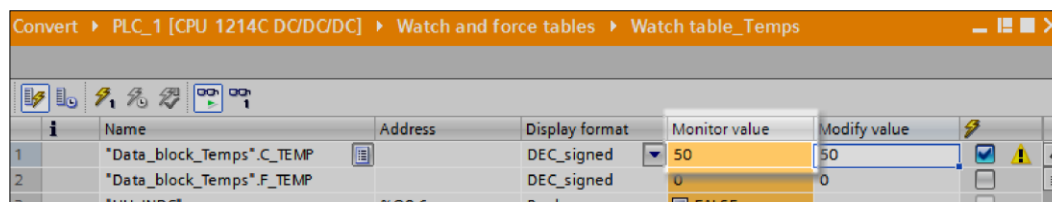
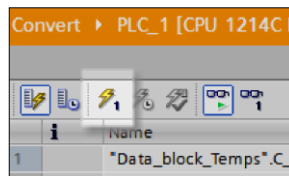
INFORMATION - TO CHANGE A TAG VALUE WITH THE WATCH WINDOW

Type in a value in the **Modify value** for "Data_block_Temps".C_TEMP.



Select the **Modify once** icon.

The value in the Data Tag has changed.



ENABLE HARDWIRED PANEL

1. Go to the **MAINTENANCE** screen.

MAINT.

2. Select **HARDWIRED PANEL ENABLED**.

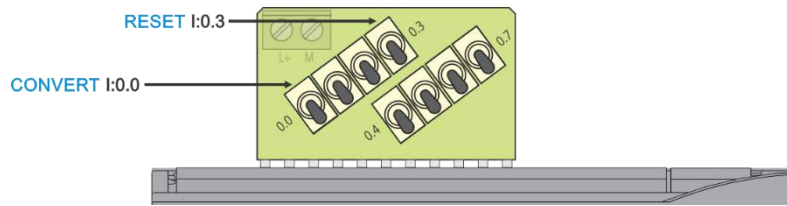
HARDWIRED PANEL ENABLED

MODIFY A VALUE AND USE THE SWITCHES

1. Set "Data_block_Temps".C_TEMP to a value of 50 in the **Watch Window**.

continue



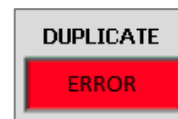


2. Turn the **switch I0.0** CONVERT to the **up** position.
3. Turn the **switch I0.0** CONVERT_PB to the **down** position.
4. Check for the correct converted temperature, **should be 122° F**.

TEST DUPLICATE ERROR

1. Do Not Change the "**Data_block_Temps**".**C_TEMP** value.
2. Turn the **switch I0.0** CONVERT to the **up** position.
3. Turn the **switch I0.0** CONVERT to the **down** position.

DUPLICATE VALUE indicator should be ON.



RESET

1. Turn the **switch I0.3** RESET to the **up** position.
2. Turn the **switch I0.3** RESET to the **down** position.

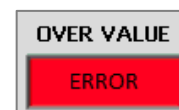
ERROR OVER VALUE indicator should be OFF.



TEST OVER VALUE ERROR

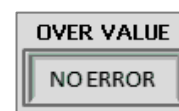
1. Set "**Data_block_Temps**".**C_TEMP** to a value of 101 in the **Watch Window**.
2. Turn the **switch I0.0** CONVERT to the **up** position.
3. Turn the **switch I0.0** CONVERT to the **down** position.

ERROR OVER VALUE indicator should be ON.



1. Turn the **switch I0.3** RESET to the **up** position.
2. Turn the **switch I0.3** RESET to the **down** position.

ERROR OVER VALUE indicator should be OFF.



EXAMPLE CODE

Momentary Pushbutton Normally Open

INDICATOR On when pushbutton pressed, that is, a logic 1

```
INDICATOR" := PUSHBUTTON";
```

Momentary Pushbutton Normally Closed

INDICATOR On when pushbutton pressed, that is, a logic 0.

```
"INDICATOR" := NOT "PUSHBUTTON";
```

Maintained Switch ON/OFF

```
If "SWITCH" then;  
  "INDICATOR" := 1;  
else;  
  "INDICATOR" := 0;  
end_if;
```

Holding Circuit

START - Normally Open

STOP - Normally Closed

```
"MOTOR" := "STOP" AND ("START" OR "MOTOR");
```

Timer On

```
"DWELL".TON(IN:= "SWITCH",  
            PT:=T#5000ms);
```

```
"INDICATOR" := "DWELL.Q"
```

Move

```
"VALUE2" := "VALUE1";
```



One Shot Rising

```
"ONE_SHOT" (CLK:="PUSHBUTTON",  
             Q=>#PULSE);  
  
"STATE" := #PULSE;  
  
If #PULSE then  
  "VALUE2" := "VALUE1";  
end_if;
```

Select Case

```
Case "SENSOR" of  
10..20:  
    "L_VALUE" := 1;  
    "M_VALUE" := 0;  
    "H_VALUE" := 0;  
21..30:  
    "L_VALUE" := 0;  
    "M_VALUE" := 1;  
    "H_VALUE" := 0;  
31..40:  
    "L_VALUE" := 0;  
    "M_VALUE" := 0;  
    "H_VALUE" := 1;  
end_case;
```

